

ĐỀ CƯƠNG THỰC TẬP: JAVA & ENTERPRISE AI SYSTEMS

System Design Principles --> RAG --> Multi-Agents

BỐI CẢNH DOANH NGHIỆP THỰC TẾ

VCCorp là một công ty công nghệ lớn sở hữu nhiều phòng ban và chi nhánh hoạt động chuyên môn hóa (Nhân sự - HR, Tài chính - Finance, Phát triển sản phẩm - R&D, và Ban Giám đốc - Board). Mỗi phòng ban đều có hàng ngàn tài liệu quy chế nội bộ, chính sách chuyên môn, cẩm nang nghiệp vụ và lịch sử vận hành riêng biệt.

Hiện tại, việc tra cứu tài liệu của nhân viên rất thủ công, làm giảm năng suất lao động. Ban Giám đốc VCCorp quyết định đầu tư xây dựng một nền tảng dùng chung có tên là **VCC Enterprise AI Knowledge Platform (VCC-EAP)** để phục vụ nội bộ công ty.

Dự án yêu cầu các tiêu chí khắt khe về **bảo mật cô lập dữ liệu cấp phòng ban (Department-level Isolation)**, khả năng **chịu tải lớn và bền vững trước thảm họa (Reliability & Concurrency)**, khả năng **tối ưu chi phí gọi AI (FinOps)**, và đặc biệt là hệ thống **đa tác tử (Multi-Agent System)** tự động kiểm soát chất lượng thông tin trả về để tránh lỗi đưa tin sai lệch gây hậu quả pháp lý cho công ty.

HƯỚNG DẪN VỀ VAI TRÒ VÀ TRÁCH NHIỆM TRONG KỲ THỰC TẬP

- Mentor (Khách hàng):** Đóng vai trò là Giám đốc Nghiệp vụ hoặc Khách hàng đại diện cho VCCorp **không biết kỹ thuật**. Khách hàng chỉ cung cấp yêu cầu dưới dạng mục tiêu kinh doanh, trải nghiệm người dùng mong muốn, ngân sách chi phí và cam kết chất lượng vận hành (SLA). Khách hàng không quan tâm bạn dùng thư viện hay code gì, chỉ quan tâm hệ thống chạy đúng và vượt qua các bài kiểm thử phá hoại (destructive testing) khi nghiệm thu.
- Student (Tech Lead / Architect):** Bạn **KHÔNG PHẢI** là người trực tiếp code (Code Monkey). Bạn đóng vai trò là Trưởng nhóm kỹ thuật tự chịu trách nhiệm. Bạn được toàn quyền sử dụng các công cụ Trí tuệ nhân tạo (AI) để hỗ trợ sinh code nhanh, nhưng bạn bắt buộc phải kiểm duyệt code và tự ra quyết định thiết kế hệ thống. Nếu code do AI sinh ra bị sập nguồn khi kiểm thử, gây rò rỉ dữ liệu hoặc vượt quá ngân sách, bạn phải chịu trách nhiệm sửa chữa và giải trình trước Khách hàng.

CHI TIẾT QUÁ TRÌNH THỰC TẬP

TUẦN 1: THIẾT KẾ CẤU TRÚC PHÂN QUYỀN HỆ THỐNG

- Cấp độ: Đơn giản

1. Bối cảnh Nghiệp vụ thô từ Khách hàng

"Tôi muốn xây dựng phần nền móng cho hệ thống VCC-EAP để sử dụng nội bộ tại công ty công nghệ VCCorp. Công ty chúng ta có nhiều phòng ban khác nhau (Nhân sự - HR, Tài chính - Finance, Phát triển sản phẩm - R&D, và Ban Giám đốc - Board). Yêu cầu tiên quyết là thông tin và tài liệu của phòng ban nào thì chỉ nhân viên của phòng ban đó được phép xem và quản lý. Riêng tài liệu của Ban Giám đốc chỉ có thành viên Ban Giám đốc được xem.

Đặc biệt, hệ thống sẽ phân loại tài liệu thành 2 loại:

- Tài liệu Gốc (Original):** Là tài liệu chứa nội dung văn bản thực tế được tải lên và lưu trữ trực tiếp trong kho tri thức của phòng ban (ví dụ: file PDF "Quy chế lương thưởng 2026.pdf" của phòng Tài chính). Tài liệu này cần được đọc, phân tích phục vụ việc tra cứu.
- Tài liệu Liên kết (Alias):** Là tài liệu không chứa nội dung thực tế mà chỉ đóng vai trò là một "đường dẫn" hoặc "lối tắt" (shortcut/link) trỏ tới một Tài liệu Gốc khác nhằm mục đích chia sẻ nhanh tri thức giữa các phòng ban mà không cần tải lại file (tránh nhân bản dữ liệu gây lãng phí đĩa). Khi truy cập Tài liệu Liên kết, hệ thống chỉ cần đọc thông tin cấu hình liên kết để điều hướng tới nội dung của Tài liệu Gốc tương ứng.

Thống kê thực tế cho thấy **99% số lượng yêu cầu** truy cập thông tin tài liệu là thuộc loại Gốc, chỉ có **1%** thuộc loại Alias. Bạn hãy thiết kế cấu trúc dữ liệu và hệ thống để tối ưu hóa việc phân loại này, đảm bảo hệ thống không bị lãng phí tài nguyên và chi phí cho việc kiểm tra loại tài liệu trong cơ sở dữ liệu."

2. Cam kết vận hành bắt buộc (SLA)

- Cô lập dữ liệu:** Khả năng rò rỉ dữ liệu chéo giữa các phòng ban khi truy vấn = **0%**.
- Tối ưu hóa tài nguyên:** Thời gian kiểm tra loại tài liệu (Original/Alias) phải tiệm cận **0ms** và không được làm tăng số lượng phép đọc đĩa (I/O) hay quét chỉ mục (Index lookup) trong cơ sở dữ liệu.
- Tính sẵn sàng API:** Có tài liệu API chuẩn hóa, hoạt động đúng 100% các luồng CRUD cơ bản.

3. Sản phẩm phải bàn giao

- ADR #001:** Tài liệu Quyết định Thiết kế Kiến trúc giải thích phương án cô lập dữ liệu cấp phòng ban và giải pháp tối ưu hóa nhận diện loại tài liệu (Original/Alias).

- Sơ đồ thực thể dữ liệu **ERD** và thiết kế phân quyền (Role-based Access Control).
 - API Swagger/OpenAPI chạy được.
 - Mã nguồn Java chạy được chức năng tạo phòng ban, người dùng và upload metadata của tài liệu.
-

TUẦN 2: XỬ LÝ TRÙNG LẬP & TẢI ĐỒNG THỜI

- **Cấp độ:** Trung bình

1. Bối cảnh Nghiệp vụ thô từ Khách hàng

"Trong thực tế, khi mạng của văn phòng bị chậm, nhân viên thường có thói quen bấm nút 'Tải lên' liên tục 4-5 lần vì nghĩ hệ thống bị treo. Tôi không muốn hệ thống của mình lưu trùng lặp 4-5 file giống hệt nhau vào tài khoản của phòng ban, gây lãng phí dung lượng đĩa cứng và tốn chi phí xử lý file trùng."

2. Cam kết vận hành bắt buộc (SLA)

- **Tính duy nhất:** 100 requests gửi lên trùng lặp cùng lúc chỉ được tạo ra **đúng 1 bản ghi** tài liệu duy nhất trong DB.
- **Nhất quán:** Không xảy ra lỗi ghi đè dữ liệu hoặc hỏng trạng thái tài liệu khi chạy tải song song.

3. Sản phẩm phải bàn giao

- **ADR #002:** Giải pháp chi tiết về cách chống trùng lặp file và xử lý ghi đồng thời.
 - Mã nguồn API tích hợp cơ chế chống trùng lặp và xác thực yêu cầu.
 - Script kiểm thử (JMeter / Apache Bench / k6) giả lập bắn 100 request upload cùng 1 file đồng thời.
-

TUẦN 3: XỬ LÝ BẤT ĐỒNG BỘ CHỊU LỖI

- **Cấp độ:** Khó

1. Bối cảnh Nghiệp vụ thô từ Khách hàng

"Việc phân tích nội dung một tài liệu PDF hàng trăm trang rất tốn thời gian. Tôi muốn nhân viên bấm upload xong là hệ thống báo 'Đã nhận file' ngay lập tức để họ đi làm việc khác, quy trình xử lý file phải chạy ngầm."

Hơn nữa, nếu máy chủ của chúng ta bị mất điện đột ngột hoặc các dịch vụ bên ngoài bị sập tạm thời trong lúc đang xử lý tài liệu ngầm, hệ thống phải tự phục hồi khi hoạt động lại, chạy tiếp các phần dở dang mà không bắt người dùng phải upload lại file."

2. Cam kết vận hành bắt buộc (SLA)

- **Tốc độ API:** API upload trả về kết quả tiếp nhận trong vòng **< 300ms**.
- **Chịu lỗi:** Tỷ lệ mất mát tài liệu đang xử lý ngầm khi server bị crash đột ngột = **0%**.

3. Sản phẩm phải bàn giao

- **ADR #003:** Giải pháp thiết kế chịu lỗi và phục hồi tiến trình ngầm sau thảm họa sập nguồn.
- Sơ đồ tuần tự (**Sequence Diagram**) mô tả luồng chạy chịu lỗi.
- Mã nguồn Java xử lý bất đồng bộ đảm bảo tính nhất quán giao dịch khi gửi tin nhắn.
- Chương trình demo khả năng tự khôi phục sau lệnh crash hệ thống (`kill -9`).

TUẦN 4: SỐ HÓA & TRA CỨU TRI THỨC CƠ BẢN (BASIC RAG)

- **Cấp độ:** Trung bình

1. Bối cảnh Nghiệp vụ thô từ Khách hàng

"Bây giờ tài liệu đã được lưu trữ an toàn, tôi muốn nhân viên của mình có thể đặt câu hỏi bằng ngôn ngữ nói tự nhiên và hệ thống tự động tìm ra chính xác những đoạn thông tin liên quan nhất để họ đọc. Ví dụ: khi nhân viên hỏi 'Tôi đi xe buýt đi làm có được trợ cấp không?', hệ thống phải tìm ra đoạn văn bản nói về 'chính sách hỗ trợ phương tiện công cộng' mặc dù hai câu này không trùng từ khóa."

2. Cam kết vận hành bắt buộc (SLA)

- **Độ chính xác truy xuất:** Top 3 kết quả trả về phải chứa trực tiếp thông tin cần tìm.
- **Thời gian tìm kiếm:** Tốc độ phản hồi tìm kiếm vector dưới **< 500ms**.

3. Sản phẩm phải bàn giao

- **ADR #004:** Giải thích rõ chiến lược cắt mảnh (chunk size, overlap) và thuật toán tìm kiếm khoảng cách vector đã chọn.
- Mã nguồn module phân tích PDF, cắt mảnh và tạo vector lưu trữ vào PostgreSQL.
- API tìm kiếm tương đồng ngữ nghĩa.

TUẦN 5: BẢO MẬT TÌM KIẾM & ĐÁNH GIÁ TỰ ĐỘNG

- **Cấp độ:** Khó

1. Bối cảnh Nghiệp vụ thô từ Khách hàng

"Tôi có hai yêu cầu đặc biệt quan trọng:

- Thứ nhất, việc tìm kiếm ngữ nghĩa không được phép vượt qua bộ lọc quyền hạn phòng ban. Nhân viên phòng Nhân sự tuyệt đối không bao giờ được tìm ra các đoạn tài liệu lương mật của Ban Giám đốc khi họ gõ các câu hỏi vu vơ.
- Thứ hai, trợ lý AI trả lời câu hỏi phải luôn đính kèm chính xác nguồn gốc (tên file, số trang) của đoạn tài liệu tham khảo để nhân viên tự kiểm chứng. Ngoài ra, hãy tự động hóa việc chấm điểm chất lượng câu trả lời của trợ lý hàng ngày để chúng ta biết chất lượng đang tốt hơn hay tệ hơn."

2. Cam kết vận hành bắt buộc (SLA)

- Rò rỉ dữ liệu:** Tỷ lệ tìm chéo dữ liệu vector của phòng ban khác = **0%**.
- Tính xác thực trích dẫn:** 100% câu trả lời có nguồn trích dẫn đúng trang tài liệu PDF gốc.

3. Sản phẩm phải bàn giao

- Mã nguồn API tìm kiếm vector tích hợp bộ lọc phòng ban và vai trò ở mức DB.
- Module tự động chạy đánh giá 50 câu hỏi mẫu (Evaluation Harness).
- Báo cáo chất lượng RAG (RAG Evaluation Report) so sánh giữa các cấu hình chunking khác nhau.

TUẦN 6: TRỢ LÝ TỰ GỌI API NGHIỆP VỤ (SINGLE AGENT & TOOLS)

- Cấp độ:** Khó

1. Yêu cầu Nghiệp vụ từ Khách hàng

"Tôi muốn trợ lý tri thức này không chỉ biết đọc tài liệu rồi đứng nhìn. Nó phải giúp nhân viên của tôi thực thi hành động nghiệp vụ thực tế. Ví dụ, khi nhân viên nói 'Đăng ký nghỉ phép cho tôi vào thứ Sáu tới', trợ lý phải tự động gọi API của hệ thống nhân sự Java để đăng ký.

Đặc biệt, câu trả lời gửi về hệ thống Java phải luôn là cấu trúc dữ liệu JSON chuẩn để hệ thống tự động xử lý được. Nếu AI trả về văn bản lỗi làm crash app Java, bạn sẽ bị phạt."

2. Cam kết vận hành bắt buộc (SLA)

- Tỷ lệ crash phân tích dữ liệu:** = **0%**. Ứng dụng Java tuyệt đối không bị văng lỗi định dạng khi AI sinh JSON hỏng.
- An toàn API:** AI không được gọi các API vượt cấp quyền phòng ban/vai trò của người

dùng hiện tại.

3. Sản phẩm phải bàn giao

- **ADR #005:** Giải pháp thiết kế an toàn API và xử lý lỗi JSON từ LLM.
- Mã nguồn Java khai báo công cụ và điều hướng gọi API.
- Module tự sửa lỗi cú pháp JSON (Self-Correction).
- *Gợi ý:* Sinh viên được phép tìm hiểu và sử dụng các công cụ/framework hỗ trợ phát triển ứng dụng AI có sẵn trên Java (ví dụ: **Spring AI** hoặc **LangChain4j**) để tối ưu hóa việc liên kết và gọi các API hệ thống (Tool Calling / Function Calling).

TUẦN 7: MẠNG LƯỚI ĐA TÁC TỬ PHỐI HỢP & BỀN VỮNG TRẠNG THÁI (MULTI-AGENT PERSISTENCE)

- **Cấp độ:** Rất khó

1. Yêu cầu Nghiệp vụ từ Khách hàng

"Khi nhân viên yêu cầu viết một báo cáo phân tích chính sách phức tạp, tôi muốn có một bộ phận chuyên tìm kiếm tài liệu, một bộ phận chuyên soạn báo cáo, và một bộ phận thẩm định độc lập kiểm tra lỗi sai để đảm bảo không có chi tiết tự bịa.
Vì quy trình phối hợp này chạy rất lâu, nếu server của bạn bị sập nguồn giữa chừng khi các bộ phận đang làm việc, hệ thống phải tự phục hồi và chạy tiếp từ đúng bước bị hỏng chứ không bắt đầu lại từ đầu khiến nhân viên của tôi phải chờ lại từ đầu."

2. Cam kết vận hành bắt buộc (SLA)

- **Khả năng khôi phục luồng Agent:** Tự động phục hồi trạng thái làm việc gần nhất của chuỗi tác tử sau sự cố sập nguồn với tỷ lệ khôi phục thành công = **100%**.
- **Tránh treo hệ thống:** Hệ thống tự động ngắt và phản hồi khi đạt giới hạn, không rơi vào vòng lặp vô hạn.

3. Sản phẩm phải bàn giao

- Mã nguồn Java điều phối hoạt động và luồng trao đổi của 4 Agent (Planner, Retriever, Writer, Critic).
- Cơ chế lưu trữ trạng thái bền vững và khôi phục luồng Agent từ Database.
- Sơ đồ Tracing nhật ký suy nghĩ và trao đổi nội bộ của các Agent.
- *Gợi ý:* Sinh viên được phép sử dụng các thư viện/framework AI tích hợp hoặc các bộ máy trạng thái có sẵn (như các tính năng **Advisors/AI Services** của LangChain4j/Spring AI, hoặc **Spring State Machine**) để quản lý luồng điều phối và lưu trữ trạng thái của tác tử.

TUẦN 8: TỐI ƯU CHI PHÍ & VẬN HÀNH HỆ THỐNG

- Cấp độ: Rất khó

1. Yêu cầu Nghiệp vụ từ Khách hàng

"Hệ thống đã hoàn thiện. Bây giờ tôi muốn đưa hệ thống vào hoạt động chính thức cho toàn bộ nhân viên công ty. Tôi cần đảm bảo:

- Tối ưu hóa chi phí sử dụng LLM: AI rất đắt đỏ, nếu nhân viên hỏi đi hỏi lại những câu hỏi tương tự nhau (ví dụ hỏi đi hỏi lại cùng một ý), hệ thống phải tự lấy câu trả lời có sẵn trong cache trả về ngay, không được gọi LLM.
- Nếu nhà cung cấp AI chính bị sập mạng hay khóa tài khoản đột ngột, hệ thống vẫn phải tự chuyển sang dịch vụ AI dự phòng để nhân viên của tôi không nhận ra gián đoạn.
- Tôi cần kiểm soát định mức token gọi AI của từng phòng ban để tránh việc một phòng ban sử dụng quá đà làm thâm hụt ngân sách chung.
- Có tài liệu thiết kế hệ thống hoàn chỉnh để đội ngũ kỹ thuật của tôi tiếp quản vận hành."

2. Cam kết vận hành bắt buộc (SLA)

- Tối ưu chi phí (Semantic Cache Hit Rate):** Thời gian phản hồi với các câu hỏi trùng lặp ý nghĩa (>95% tương đồng) giảm xuống < **50ms** và chi phí gọi LLM API của lượt hỏi đó bằng \$0.
- downtime khi sập AI:** downtime tự động chuyển mô hình dự phòng < **5 giây**.
- Định mức chi phí:** Khóa quyền gọi AI của tài khoản/phòng ban lập tức nếu vượt quá định mức chi phí (Token quota) đã đăng ký trong tháng.

3. Sản phẩm phải bàn giao

- Mã nguồn cơ chế bộ đệm ý nghĩa (Semantic Cache) và cơ chế chuyển đổi nóng (Hot Failover) nhà cung cấp AI.
- Module giám sát chi phí token và giới hạn Quota theo từng phòng ban.
- Tài liệu Dự án Cuối kỳ hoàn chỉnh** bao gồm toàn bộ sơ đồ kiến trúc (ERD, Component, Sequence, Tracing Diagram) và các ADRs từ #001 đến #006.